DISTRIBUTED CRYPTOGRAPHIC SEARCH FRAMEWORK

Applied Cryptography Study

1. Overview

An MPI-based distributed cryptographic search framework was built to establish a reusable foundation for later cryptographic studies. First, a manager-worker MPI architecture is established across cluster nodes, and then brute-force and table-assisted search workflows are supported through a pluggable verifier model. As used herein, a node refers to an individual machine within the cluster, whereas a worker refers to a processor core.

2. Experimental Setup

To perform the workflows, Raspberry Pi 5 clusters were constructed. The clusters were constructed with a dedicated head node for scheduling and compute nodes for computation. Parallelization was achieved using MPI and implemented using Python and mpifort.

2.1 Hardware Layer

To perform the studies herein, two small clusters were constructed consisting of Raspberry Pi 5 ("RP5") nodes, each equipped with a Waveshare PoE Hat (F), allowing both power and Gigabit Ethernet connectivity via a single cable. Each RP5 featured a quad-core ARM Cortex-A76 CPU operating at approximately 2.4 GHz, thereby providing four processing cores per compute node. Each system utilized a SanDisk 128GB Extreme microSDXC UHS-I microSD card for storage.

The two clusters had different architectures. The cluster used to establish and validate the manager-worker MPI architecture consisted of one head node (manager) and three compute nodes. In this configuration, computational workflow was distributed across 12 workers (4 per compute node). Due to budgetary constraints, the head node and one compute node had 8 GB RAM, while the remaining two compute nodes had 4 GB RAM. This limited heterogeneity in memory capacity did not affect validation of the manager-worker MPI architecture, because the corresponding workloads were intentionally designed to be CPU-bound rather than memory-bound. As a result, performance depended primarily on processor characteristics rather than memory capacity, and the impact of this heterogeneity was minimal.

For reasons explained below, the cryptographic calculations were performed on a cluster consisting of one head node (manager) and four compute nodes, all with 8 GB RAM. In this configuration, computational workflow was distributed across 16 workers (4 per compute node). This revised configuration was adopted to support more demanding table-assisted cryptographic workloads without the memory constraints encountered on the resource-constrained

cluster used to establish and validate the manager-worker MPI architecture.

A separate control server was used both to access the cluster nodes and to provide shared storage services across all nodes.

2.2 Software Layer

The cluster was configured to implement a manager-worker MPI architecture with dynamic workload distribution. Each node was running PiOS kernel version 6.12 and Open MPI version 5.0.7. The nodes were running in a headless configuration and accessed via network from a control server running Ubuntu 25.10 (AMD 64-bit). The control server also provided an NFS export to each of the compute nodes. The common share provided each node access to runtime files used for the computations. Notably, because network performance and disk I/O were relevant to the table-assisted workflows, all nodes were connected to the same switch on an isolated VLAN through wired Gigabit Ethernet. In this configuration, traffic between nodes remained on the local switch backplane and did not traverse the gateway.

The manager-worker MPI architecture runtimes were implemented in Python 3.13.5 within a virtual environment for consistency and isolation across all nodes. Brute-force and table-assisted runtimes were compiled with mpifort 14.2.0.

3. Implementation of Manager-Worker MPI Architecture

The manager-worker architecture was built using the RP5 cluster, having four nodes: a manager node and 3 worker nodes. The implementation was verified, and its effectiveness was demonstrated using a purposefully designed workload distribution function.

3.1 Validation and Effectiveness of Dynamic Workload Distribution

The effectiveness of dynamic workload distribution was evaluated by computing workload functions with varying computational cost and comparing time-to-completion under both static and dynamic MPI scheduling. Validation was performed using a benchmark workload function, $K(x)$, constructed as:

$$K(x) = \int_0^a f(x)(1 + S(x))\, dx \tag{3.1}$$

where the baseline workload kernel, $f(x)$, is given by:

$$f(x) = \sqrt{x} \tag{3.2}$$

and the workload perturbation function, $S(x)$, is given by:

$$S(x) = \begin{cases} 0, & kernel = plain \\ \epsilon \sum_{k=1}^{N(x)} \sin(x + \alpha k)\cos(x + \beta k), & kernel = variable \end{cases} \tag{3.3}$$

$$N(x) = \begin{cases} I_1 & 0 \le x < \frac{a}{3} \\ I_2 & \frac{a}{3} \le x < \frac{2a}{3} \\ I_3 & \frac{2a}{3} \le x \le a \end{cases} \tag{3.4}$$

and where

$$\begin{aligned} a &= 10^8 \\ [I_1, I_2, I_3] &= (5, 50, 500) \\ \epsilon &= 10^{-12} \\ \alpha &= 10^{-6} \\ \beta &= 2 \times 10^{-6} \end{aligned} \tag{3.5}$$

By ensuring that its value is effectively invariant to the input-dependent computational cost (eq.3.3), the benchmark workload function validates the scheduling framework. More particularly, the benchmark workload function isolates the impact of MPI scheduling by ensuring that any observed variations are attributable to the scheduling strategy.

From Eq. (3.3), it can be seen that $|S(x)| \le \epsilon N(x)$, because each term in the summation, $\sin(x + \alpha k)\cos(x + \beta k)$, lies in the interval $[-1, 1]$. The parameter ϵ is chosen such that $\epsilon \max(N(x)) \ll 1$, ensuring that $|S(x)| \ll 1$ over the domain of interest. Consequently, the modified integrand $f(x)(1 + S(x))$ is effectively equivalent to the baseline kernel $f(x)$, and the benchmark workload function (Eq. (2.1)) remains effectively invariant, with a value approximately equal to the baseline reference value $\frac{2}{3}a^{3/2}$. This invariance depends solely on the choice of a and ϵ, both of which are fixed for each of the calculations below.

The term $\sin(x + \alpha k)\cos(x + \beta k)$ incorporates a phase shift so that the summation is not trivial, that is, consisting of identical terms over $N(x)$. Accordingly, we chose $\beta = 2\alpha$, such that α is small, thereby ensuring distinct phase increments in k while keeping the perturbation numerically smooth and bounded.

The parameter a, the upper limit of the benchmark workload function, determined the total workload size. The parameter a was empirically selected to ensure sufficiently long runtimes

while making scheduling effects measurable and stable. At the same time, the value of a was selected to mitigate issues related to startup overhead and MPI communication latency, while keeping $K(x)$ values numerically well behaved.

The parameters I_1, I_2, and I_3 were empirically selected to impose three clearly separated workload classes with approximate relative cost ratios of 1:10:100. This choice was made to produce a computational imbalance large enough to be observable under static scheduling while remaining simple to interpret.

3.2 Results and Discussion

The results were obtained by computing $K(x)$ as described above. MPI was configured such that the head node (manager) was constrained to a single MPI slot in order to prevent it from participating in computational workloads. Consequently, the manager process was dedicated to scheduling on the head node, while the remaining 12 workers spanning the 3 compute nodes were dedicated to computation. The numerical evaluation of $K(x)$ was performed on a grid of a points. Under static scheduling, the grid was partitioned into 12 fixed assignments, so that each worker received approximately $a/12$ evaluations over a single contiguous interval in x. Under dynamic scheduling, the same grid was partitioned into 100 chunks, each containing $a/100$ evaluations, which were assigned to workers on demand. Consequently, the total number of evaluations in both static and dynamic scheduling was the same: a.

From Table (3.1) it can be seen that the effects of varying the load under static scheduling are predictable and demonstrate the validity of the MPI architecture. For the plain kernel, the total compute time was 1.95 s. Over the number of compute nodes, μ was 1.82 s and σ was 0.0190 s. The relatively low σ reflects the fact that for the plain kernel, the compute load is essentially the same for each worker (about $a/12$ per worker).[1] The accuracy of scheduling can be confirmed by the numerical integration, which showed a relative error of about 1.11×10^{-13}, with respect to the analytical solution of $\frac{2}{3}a^{\frac{3}{2}}$.

Table 3.1: Static Scheduling Summary Metrics

Kernel	Numerical Integral	Absolute Error	Relative Error	Total Time (s)
Plain	6.67×10^{11}	0.0739	1.11×10^{-13}	1.95 $\mu = 1.82$ s, $\sigma = 0.0190$ s
Variable	6.67×10^{11}	0.0642	9.63×10^{-14}	876 $\mu = 320$ s, $\sigma = 384$ s

For the variable kernel, the total compute time and the mean per-worker compute time rose dramatically to 876 s and 384 s, respectively. These results were expected because the load on each worker was increased dramatically due to the perturbation terms in Eq. (3.3). σ also dramatically increased, as the total load on each compute node, and its corresponding workers, were significantly different. This can be clearly seen by time per rank in Table 3.2.

Table 3.2: Static Scheduling Performance Comparison by Cost Region

Rank	Node	Steps	Interval	Time (Plain) [s]	Time (Variable) [s]
Low-cost region ($N(x) = 5$)					
1	Node 1	8333334	[0.0, 8.33e6]	1.80	12.4
2	Node 1	8333334	[8.33e6, 1.67e7]	1.81	12.7
3	Node 1	8333334	[1.67e7, 2.50e7]	1.81	12.7
4	Node 1	8333334	[2.50e7, 3.33e7]	1.81	12.4
Mid-cost region ($N(x) = 50$)					
5	Node 2	8333333	[3.33e7, 4.17e7]	1.84	86.9
6	Node 2	8333333	[4.17e7, 5.00e7]	1.87	86.9
7	Node 2	8333333	[5.00e7, 5.83e7]	1.82	86.7
8	Node 2	8333333	[5.83e7, 6.67e7]	1.82	88.0
High-cost region ($N(x) = 500$)					
9	Node 3	8333333	[6.67e7, 7.50e7]	1.82	862
10	Node 3	8333333	[7.50e7, 8.33e7]	1.83	861
11	Node 3	8333333	[8.33e7, 9.17e7]	1.81	865
12	Node 3	8333333	[9.17e7, 1.00e8]	1.81	876

As a function of Node number, the time per worker increased in 3 blocks: Node 1 $<$ Node 2 $<$ Node 3. This result is a direct consequence of the choice of $I_1 - I_3$ in eqns. (3.3) and (3.4). The workload along x in Eq. (3.1) is sequentially distributed over 12 workers and I is distributed over the range x in $\frac{a}{3}$ segments. Consequently, compute Nodes 1, 2 and 3 each compute the variable kernel (Eq. (3.3)) sequentially over I_1, I_2, and I_3, respectively. Because the computational cost increases with increasing I_n index, it can be seen from Table (3.2) that three regions are formed: Node 1 with low computational cost, Node 2 with moderate computational cost, and Node 3 with high computational cost.

While the computational cost scales approximately by an order of magnitude with the I_n index, discrepancies are seen at lower values of I_n, due to fixed per-step overhead. As the I_n index increases, the scaling approaches expected values. For example, the compute times for Node 2 are slightly less than 10 times those for Node 1, while compute times for Node 3 are about the expected 10 times of those for Node 2.

As was the case with the plain kernel, the accuracy of scheduling under the variable kernel was confirmed by the numerical integration, which showed a relative error of about

9.63×10^{-14}. This confirms that $S(x)$, Eq. (3.3), is behaving as intended. Namely, $S(x)$ is significantly increasing the computational cost of $K(x)$ without appreciably altering its value. The reduced relative error observed for the variable kernel with respect to the plain kernel is due to incidental cancellation effects introduced by the oscillatory perturbation and does not reflect improved numerical accuracy.

Dynamic scheduling further validates the accuracy and implementation of the MPI architecture. More particularly, the results demonstrate that load-balancing is functioning as intended, without affecting the accuracy of the numerical value of the benchmark workload function (Eq. (3.1)). Table (3.3) shows that while the total compute times for the fixed kernel were similar to those obtained with static scheduling, dynamic scheduling reduced the total compute time of the benchmark workload function by about 61%.

Table 3.3: Dynamic Scheduling Summary Metrics

Kernel	Numerical Integral	Absolute Error	Relative Error	Total Time (s)
Plain	6.67×10^{11}	0.0636	9.54×10^{-14}	2.00
				$\mu = 1.82$ s, $\sigma = 0.0957$ s
Variable	6.67×10^{11}	0.0427	6.41×10^{-14}	346
				$\mu = 320$ s, $\sigma = 39.7$ s

The similarities in the total compute time between static and dynamic scheduling for the plain kernel reflect the fact the workload per worker is the same in each. For static scheduling, each worker simultaneously gets contiguous $\frac{a}{12}$ assignments along x, each with the same cost ($\sqrt{x}\frac{a}{12}$). Consequently, load balancing via dynamic scheduling does not appreciably improve computational efficiency. On the other hand, for static scheduling under the variable kernel, each worker again simultaneously gets $\frac{a}{12}$ assignments contiguous along x, however, the computational cost varies with each assignment according to the perturbation term in Eq. (3.3). This leads to idle workers during the evaluation of the benchmark function (see variable times in Table (3.2)). In this scenario, dynamic scheduling provides effective load balancing because the variable workload is distributed relatively evenly over each worker. This can be more clearly seen by comparing the timing data in Tables (3.1) and (3.3). As can be seen in the tables, the total compute times for the plain kernel were similar under both static and dynamic scheduling. Moreover, in both cases, the μ was identical and σ was small relative to the total compute times.[2] This can be more clearly seen in Table (3.4).

Referring to the table, times for the plain kernel computation per worker were similar to the times under static scheduling. For the variable kernel, times were significantly more evenly distributed relative to the times under static scheduling. Put another way, when comparing the variable kernel times, dynamic scheduling mitigates against idle workers that would

Table 3.4: Dynamic Scheduling Cumulative Per-Rank Time Comparison

Rank	Node	Time (Plain) [s]	Time (Variable) [s]
		Node 1	
1	Node 1	1.77	344
2	Node 1	1.74	273
3	Node 1	1.95	336
4	Node 1	1.74	336
		Node 2	
5	Node 2	1.95	344
6	Node 2	1.74	346
7	Node 2	1.95	243
8	Node 2	1.78	345
		Node 3	
9	Node 3	1.74	345
10	Node 3	1.75	345
11	Node 3	1.95	243
12	Node 3	1.75	342

have been present under static scheduling. It is noted that workers 2 and 11 in Table (3.4) under the variable kernel appear to be outliers. These values are due to tail latency, and not unevenly distributed workload as in the case of the worker times for the variable kernel under static scheduling.

4. Brute-Force and Table-Assisted Search Workflows Supported Through a Pluggable Verifier Model

4.1 Architectural Modification

As previously mentioned in 2.1, the cluster architecture was modified to perform cryptographic calculations. The initial architecture consisted of a head node and three compute nodes; however, it limited the scope of the cryptographic calculations. More particularly, when the cryptographic workload was expanded to support more meaningful table-assisted search evaluations, the 4 GB memory capacity of two of the compute nodes proved insufficient and led to memory pressure during execution.[3] To address this limitation, the two 4 GB compute nodes were replaced with 8 GB compute nodes and one of the displaced 4 GB nodes was then repurposed as a dedicated head node (scheduling and coordination tasks impose comparatively modest memory demands). Under the revised architecture, the cluster comprised one head node and four compute nodes, providing 16 workers for computational workloads.

Importantly, the architecture revision does not alter the validity of the earlier MPI implementation results. The prior section established that the distributed manager-worker model operated correctly on the original cluster, including task dispatch, worker coordination, message passing, and termination behavior. The subsequent hardware modification was motivated by memory scalability considerations associated with larger table-assisted workloads, rather than by any flaw in the MPI architecture itself.[4]

4.2 Calibration

Prior to performing cryptographic calculations, the program was calibrated to achieve reasonable run times. Calibration, and subsequent cryptographic calculations, were performed

with MPI using static scheduling. For the cryptographic calculations, SHA256 is used to simulate the cryptographic workloads associated with WPA2 handshakes. In this scenario, the distribution of brute-force and table-assisted calculations results in cost-equivalent workloads for each worker. As seen in Section 3.2, there is effectively no efficiency gained with dynamic scheduling in such a situation.

The key space is defined by the character alphabet, candidate length, and the particular cryptographic primitive (e.g., SHA-256 in the present study). The key space is given by:

$$|\mathcal{K}| = N^L, \tag{4.1}$$

where L is the string length over an alphabet of size N. Using $N = 94$ (the full printable ASCII) and a modest string length of 12, the key space is 4.76×10^{23} and would take approximately 3.34 billion years to exhaustively search using the cluster architecture described above.[5] Accordingly, the key space was calibrated for the architecture so that reasonable runtimes could be achieved.

Calibration was done by searching over the domain

$$P = \{(N, L) \in \mathbb{Z}^2 \mid N = 9, L \in [\![4, 10]\!]\} \tag{4.2}$$

The key space alphabet was selected as ASCII codes 97 through $96 + N$ and the target was chosen as a repetitive sequence of L number of ASCII code $96 + N$. For example, for the element $(N, L) = (9, 5)$, the target key was "iiiii." In this manner, the full key space was searched at each element of the domain. The plot of time as a function of key space was then plotted and fit to a linear function ("calibration curve"):

$$t = m|\mathcal{K}| + K_0 \tag{4.3}$$

where $m = 1.43 \times 10^{-6}$, $K_0 = 0.869$ and $R^2 = 1.00$. The linear scaling with key space was expected, due to the fact that each workload segment had an equivalent, fixed cost and increasing $|\mathcal{K}|$ served only to uniformly increase the size of each workload segment.

4.3 Results and Discussion

Cryptographic calculations were run for brute-force and table-assisted searches over the domain

$$P = \{(N, L) \in \mathbb{Z}^2 \mid (N, L) \in (\{7, 8, 10, 11\} \times \{8\}) \cup \{(7, 9)\}\}. \tag{4.4}$$

For table-assisted calculations, two sets of runs were performed for each element in the domain. Candidates were partitioned into hash-prefix buckets according to the first p hexadecimal characters of the SHA-256 digest. For $p = 1$, the key space admits a partition into $16^1 = 16$ hash-prefix buckets and, for $p = 2$, into $16^2 = 256$ hash-prefix buckets. In each case, a single partition corresponding to the target key was selected as the table-assisted dataset.[6] As in the calibration runs, the key space alphabet was selected as ASCII codes 97 through $96 + N$, and the target key was chosen as a repetitive sequence of length L consisting of ASCII code $96 + N$, so as to ensure a full key space search for each element of the domain (at least for the brute-force calculations). Furthermore, initial runs for $p = 1$ and $p = 2$ showed significant noise attributable to start-up and other fixed overhead costs. Accordingly, 5 to 10 evaluations of $p = 1$ and $p = 2$ were performed at each point in the domain P, and the mean and standard deviation are reported. For brute-force calculations, run times were sufficiently long that the hashing workload dominated start-up and other computational overhead. Accordingly, brute-force calculations were evaluated once for each element in the domain P.

Timing results for the calculations can be seen in Table (4.1). Referring to the table, as expected, run times were significantly reduced using the table-assisted approach. Relative to the brute-force approach, the $p = 1$ table-assisted run times yielded a monotonically increasing 98.5% to 99.2% reduction. The $p = 2$ table-assisted runs also showed a significant reduction in compute times, monotonically increasing from 99.0% to 99.9%.

Figure (4.1) shows the run times as a function of key space. The results show the expected behavior. For each approach, the run time scales approximately linearly with $|\mathcal{K}_{BF}|$. These results are not surprising. For each element of (N, L) in P, there is a constant key space reduction for $p = 1$ and $p = 2$, with the latter being larger than the former.

The key space reduction is given by:

Table 4.1: Brute-force and table-assisted search times for each experimental run. For TA_1 and TA_2, repeated measurements are reported as mean ± standard deviation.

Run	N	L	Target Key	BF (s)	TA_1 (s)	TA_2 (s)
1	7	8	`gggggggg`	9.29	0.145 ± 0.0157	0.0884 ± 0.00445
2	8	8	`hhhhhhhh`	25.1	0.269 ± 0.00931	0.0927 ± 0.0135
3	7	9	`ggggggggg`	59.1	0.562 ± 0.0233	0.111 ± 0.0123
4	10	8	`jjjjjjjj`	145	1.25 ± 0.0170	0.146 ± 0.0134
5	11	8	`kkkkkkkk`	308	2.60 ± 0.0214	0.236 ± 0.00891

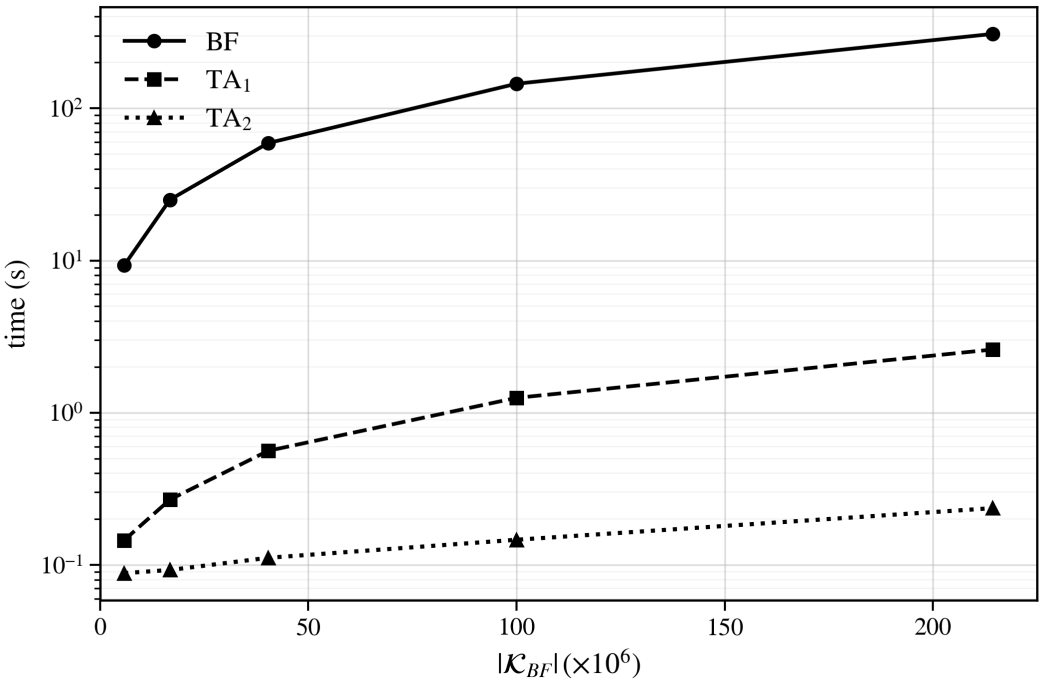


Figure 4.1: Brute-force and table-assisted runtimes as a function of $|\mathcal{K}_{BF}|$.

$$\rho_n = \frac{|\mathcal{K}_{BF}|}{|\mathcal{K}_{TA_n}|}. \tag{4.5}$$

The key space reduction can be seen in Table (4.2). For ρ_1 and ρ_2, the values are approximately 16 and 256, respectively, and remain substantially constant for each element (N, L). Accordingly, the table-assisted workflows are expected to scale proportionally with the brute-force workflow. What is particularly interesting, however, is that the times calculated from the table-assisted workflows are roughly an order of magnitude smaller than the predicted values based upon the calibration curve for the brute-force workflow.

The results show a relatively systematic difference between the predicted and calculated

Table 4.2: Brute-force keyspace sizes, reduction ratios, and predicted versus calculated table-assisted runtimes.

Run	$\lvert\mathcal{K}_{BF}\rvert$	ρ_1	$t_{\mathrm{TA}_1,\mathrm{pred}}$	$t_{\mathrm{TA}_1,\mathrm{calc}}$	ρ_2	$t_{\mathrm{TA}_2,\mathrm{pred}}$	$t_{\mathrm{TA}_2,\mathrm{calc}}$
1	5764801	16.0	1.38	0.145	258	0.901	0.0884
2	16777216	16.0	2.37	0.269	254	0.963	0.0927
3	40353607	16.0	4.48	0.562	256	1.09	0.111
4	100000000	16.0	9.81	1.25	256	1.43	0.146
5	214358881	16.0	20.0	2.60	256	2.07	0.236

times, with the predicted values exceeding the calculated values by roughly an order of magnitude. This systematic difference is more pronounced for the table-assisted calculations corresponding to $p = 2$. The principal conclusion, then, is that the brute-force calibration cannot provide a direct predictive model for the table-assisted approach, even when accounting for the reduced key space. These results indicate that the table-assisted approaches incur substantially smaller per-candidate cost than that of the brute-force approach. This rather curious result, at least from a theoretical standpoint, suggests that these differences may perhaps be partially mitigated by code-level optimization. However, that question lies well beyond the scope of the present study and instead suggests a possible direction for future work.

5. Conclusion

An MPI-based distributed cryptographic search framework was successfully developed and validated on a Raspberry Pi 5 cluster. The manager-worker MPI architecture was shown to operate correctly under both static and dynamic workload distribution, and its effectiveness was verified using a benchmark workload function with controllable computational imbalance. In particular, dynamic scheduling was shown to substantially reduce time-to-completion for uneven workloads while preserving numerical accuracy.

Building on this framework, brute-force and table-assisted cryptographic search workflows were implemented through a pluggable verifier model. Brute-force calibration demonstrated the expected linear scaling of runtime with key space size over the calibrated domain. The table-assisted workflows likewise exhibited the expected monotone scaling behavior, with substantially reduced runtimes relative to brute force. Moreover, increasing the hash-prefix length from $p = 1$ to $p = 2$ produced the expected additional reduction in effective search space and corresponding reduction in runtime.

At the same time, the results showed that the brute-force calibration curve does not directly predict the table-assisted runtimes, even after accounting for reduced key space. This indicates that the brute-force and table-assisted workflows differ not only in search-space size but also in effective per-candidate computational cost. While this prevents a strictly like-for-like runtime comparison between the two approaches, it does not alter the central result that the framework correctly supports both workflows and that the table-assisted searches provide substantial practical reductions in runtime.

Overall, this study establishes a reusable distributed framework for future cryptographic experiments on resource-constrained cluster hardware. Future work could extend the framework to more realistic table-assisted datasets, additional cryptographic primitives, and further code-level optimization aimed at bringing the per-candidate costs of brute-force and table-assisted workflows into closer agreement.

Notes

[1]In instances where the integration grid was not evenly divisible by 12, up to 11 workers had one more evaluation than the remaining nodes. This additional compute cost is negligible relative to the individual compute times of each worker.

[2]Though σ for the variable kernel under dynamic scheduling was appreciably higher than that under static scheduling, σ is still small relative to the total compute time. Furthermore, the increased compute times for dynamic scheduling under the plain kernel were due to increased overhead associated with the increased number of assignment transfers.

[3]Memory artifacts during runs of meaningful table-assisted calculations prevented full execution.

[4]Reporting this limitation is important for two reasons. First, it clarifies that the observed failure was not attributable to an error in the MPI manager-worker implementation, which had already been validated, but rather to insufficient memory capacity on the 4 GB worker nodes during larger table-assisted runs. Second, documenting the failure provides a practical reference for others attempting similar deployments on resource-constrained Raspberry Pi clusters, where replicated in-memory table structures can become the dominant scaling bottleneck.

[5]The calculation approximates the 16 core throughput of the 16 combined RP5 compute cores as 4.52 MH/s, though this value can vary significantly based upon the cryptographic primitive(s).

[6]While the selection of a single partition does not represent a realistic approach to real-world workloads, it was chosen here to isolate the effects of key space reduction and the corresponding effect on cryptographic calculations.